Durham University

Topic 3： NumPy and Pandas – cont.

Weeks 3 and 4

Igor Razgon
igor.razgon@durham.ac.uk

Content

1. Columns and index of DataFrame
2. Accessing individual values, rows, and columns. Series class
3. Concluding remarks
4. Files .csv and Excel

Durham University

Columns and index of DataFrame

columns and *index* attributes of DataFrames

- The names of the columns are stored in the *columns* attribute of a *DataFrame* instance.
- The numbers indexing the rows are stored in the *index* attribute of *DataFrame* instance.
- Both attributes can be accessed directly and printed.

Example: *columns* and *index* attributes

```
import pandas as pd

marks={
'id': [11,41,52,8],
'algebra':[60,90,70,55],
'analysis':[40,80,80,70],
'logics':[90,80,90,80]
}

pdmarks=pd.DataFrame(marks)
print('The columns are')
print(pdmarks.columns)
print('The rows are indexed with')
print(pdmarks.index)
```

The output

```
The columns are
Index(['id', 'algebra', 'analysis', 'logics'], dtype='object')
The rows are indexed with
RangeIndex(start=0, stop=4, step=1)
```

Index and *RangeIndex* types of Pandas

- Both *columns* and *index* attributes are of type *Index*.
- If any of them is an interval of integers, then the type is *RangeIndex*.
- These are list like types but *immutable*: individual elements cannot be changed.
- Yet individual elements can be accessed and printed.

Example: *Index* and *RangeIndex* types

```
import pandas as pd

marks={
'id': [11,41,52,8],
'algebra':[60,90,70,55],
'analysis':[40,80,80,70],
'logics':[90,80,90,80]
}

pdmarks=pd.DataFrame(marks)
print('The type of columns is')
print(type(pdmarks.columns))
print('The type of index is')
print(type(pdmarks.index))
print('Individual column name',pdmarks.columns[0])
```

The output

```
The type of columns is
<class 'pandas.core.indexes.base.Index'>
The type of index is
<class 'pandas.core.indexes.range.RangeIndex'>
Individual column name id
```

Durham University

Use the *columns* attribute for filtering

- The *columns* attribute provides an alternative way to **columns filtering** (in addition to the one we considered last time)
- If we want to remove IDs, run:

  exclfirst=pdmarks.columns[1:]

- Then create a new DataFrame, run

  pdmarksonly=pdmarks[exclfirst]

Example: *Exclusion of IDs from the DataFrame of student marks*

```
import pandas as pd

marks={
'id': [11,41,52,8],
'algebra':[60,90,70,55],
'analysis':[40,80,80,70],
'logics':[90,80,90,80]
}

print("\nDataFrame of student marks:")
pdmarks=pd.DataFrame(marks)
print(pdmarks)

print("\nExclusion of IDs from the DataFrame:")
exclfirst=pdmarks.columns[1:]
pdmarksonly=pdmarks[exclfirst]
print(pdmarksonly)
```

The output

```
DataFrame of student marks:
   id  algebra  analysis  logics
0  11       60        40      90
1  41       90        80      80
2  52       70        80      90
3   8       55        70      80

Exclusion of IDs from the DataFrame:
   algebra  analysis  logics
0       60        40      90
1       90        80      80
2       70        80      90
3       55        70      80
```

Durham University

Modification of columns and index

- Copy the attribute to a list.
- Change the list as you want.
- Copy the list back to the attribute.
- **Remark:** the length of the list cannot be changed as a result of the modification. Otherwise, Python will complain about a mismatch.

Example: Modification of columns

```
import pandas as pd

marks={
'id': [11,41,52,8],
'algebra':[60,90,70,55],
'analysis':[40,80,80,70],
'logics':[90,80,90,80]
}

print("\nDataFrame of student marks:")
pdmarks=pd.DataFrame(marks)
print(pdmarks)

print("\nModification of the column: algebra => algorithms")
collist=pdmarks.columns.to_list()
collist[1]='algorithms'
pdmarks.columns=pd.Index(collist)
print(pdmarks)
```

The output

```
DataFrame of student marks:
   id  algebra  analysis  logics
0  11       60        40      90
1  41       90        80      80
2  52       70        80      90
3   8       55        70      80

Modification of the column: algebra => algorithms
   id  algorithms  analysis  logics
0  11          60        40      90
1  41          90        80      80
2  52          70        80      90
3   8          55        70      80
```

Example: Modification of index

```
import pandas as pd

marks={
'id': [11,41,52,8],
'algebra':[60,90,70,55],
'analysis':[40,80,80,70],
'logics':[90,80,90,80]
}

print("\nDataFrame of student marks:")
pdmarks=pd.DataFrame(marks)
print(pdmarks)

print("\nModification of indexes: 0-3 => 20-23")
indlist=pdmarks.index.to_list()
indlistincr=[x+20 for x in indlist]
pdmarks.index=pd.Index(indlistincr)
print(pdmarks)
```

The output

```
DataFrame of student marks:
   id  algebra  analysis  logics
0  11       60        40      90
1  41       90        80      80
2  52       70        80      90
3   8       55        70      80

Modification of indexes: 0-3 => 20-23
    id  algebra  analysis  logics
20  11       60        40      90
21  41       90        80      80
22  52       70        80      90
23   8       55        70      80
```

Durham University

Durham University

Accessing individual values, rows, and columns

Series class

Access to an individual element

- Let *df* be a *DataFrame* instance.
- Let *c* be an entry of *columns* and *i* be an entry of *index*.
- *df[c][i]* gives access to the element of *df* corresponding to *c* and *i.*
- **Important**: Note that, unlike in the ordinary Python, we first specify the column and then the row.

Example: Example of access to an individual element

```
import pandas as pd

marks={
'id': [11,41,52,8],
'algebra':[60,90,70,55],
'analysis':[40,80,80,70],
'logics':[90,80,90,80]
}

print("\nDataFrame of student marks:")
pdmarks=pd.DataFrame(marks)
print(pdmarks)

print("\nModification of an individual entry: algebra grade")
pdmarks['algebra'][1]=100
print(pdmarks)
```

The output

```
DataFrame of student marks:
   id  algebra  analysis  logics
0  11       60        40      90
1  41       90        80      80
2  52       70        80      90
3   8       55        70      80

Modification of an individual entry: algebra grade
   id  algebra  analysis  logics
0  11       60        40      90
1  41      100        80      80
2  52       70        80      90
3   8       55        70      80
```

Durham University

The *Series* class

- *Series* is a class of Pandas that is used for analysis of one-dimensional data sets.
- Can be created from a Python list.
- Individual elements can be accessed analogously to Python lists.
- Has the associated *index* attribute, but does not have the *columns* attribute.

Example: series defined from a list and entry modification

```python
import pandas as pd

print("Series")
exseries=pd.Series([1,7,-1,8,15])
print(exseries)

print("\nEntry modification")
exseries[1]=100
print(exseries)
```

The output

```
Series
0     1
1     7
2    -1
3     8
4    15
dtype: int64

Entry modification
0      1
1    100
2     -1
3      8
4     15
dtype: int64
```

Individual rows and columns of a *DataFrame* as *Series*

- Individual columns and rows of a data can be accessed as series.
- Let *df* be a *DataFrame* instance, *c* be a *columns* entry, *i* be an *index* entry.
  - *df[c]* gives access to column *c*.
  - *df.loc[i]* gives access to row *i*.

Example: accessing a row and a column of a DataFrame

```
import pandas as pd

marks={
'id': [11,41,52,8],
'algebra':[60,90,70,55],
'analysis':[40,80,80,70],
'logics':[90,80,90,80]
}

print("\nDataFrame of student marks:")
pdmarks=pd.DataFrame(marks)
print(pdmarks)

print("\nAccessing column algebra")
indcolumn=pdmarks['algebra']
print(indcolumn)
print("Type is ",type(indcolumn))

print("\nAccessing row #1")
indrow=pdmarks.loc[1]
print(indrow)
print("Type is ",type(indrow))
```

The output

```
DataFrame of student marks:
   id  algebra  analysis  logics
0  11       60        40      90
1  41       90        80      80
2  52       70        80      90
3   8       55        70      80

Accessing column algebra
0    60
1    90
2    70
3    55
Name: algebra, dtype: int64
Type is  <class 'pandas.core.series.Series'>

Accessing row #1
id          41
algebra     90
analysis    80
logics      80
Name: 1, dtype: int64
Type is  <class 'pandas.core.series.Series'>
```

Example: Iteration along rows and columns

```python
import pandas as pd

marks={
'id': [11,41,52,8],
'algebra':[60,90,70,55],
'analysis':[40,80,80,70],
'logics':[90,80,90,80]
}

print("\nDataFrame of student marks:")
pdmarks=pd.DataFrame(marks)
print(pdmarks)

print('Iteration along rows')
for ind in pdmarks.index:
  print("\tRow=",ind)
  print(pdmarks.loc[ind])

print('Iteration along columns')
for col in pdmarks.columns:
  print("\tColumn=",col)
  print(pdmarks[col])
```

The output

```
DataFrame of student marks:
   id  algebra  analysis  logics
0  11       60        40      90
1  41       90        80      80
2  52       70        80      90
3   8       55        70      80
Iteration along rows
    Row= 0
id          11
algebra     60
analysis    40
logics      90
Name: 0, dtype: int64
    Row= 1
id          41
algebra     90
analysis    80
logics      80
Name: 1, dtype: int64
    Row= 2
id          52
algebra     70
analysis    80
logics      90
Name: 2, dtype: int64
    Row= 3
id           8
algebra     55
analysis    70
logics      80
Name: 3, dtype: int64
```

```
Iteration along columns
    Column= id
0    11
1    41
2    52
3     8
Name: id, dtype: int64
    Column= algebra
0    60
1    90
2    70
3    55
Name: algebra, dtype: int64
    Column= analysis
0    40
1    80
2    80
3    70
Name: analysis, dtype: int64
    Column= logics
0    90
1    80
2    90
3    80
Name: logics, dtype: int64
```

Durham University

Example: filtering using a single column

```python
import pandas as pd

marks={
'id': [11,41,52,8],
'algebra':[60,90,70,55],
'analysis':[40,80,80,70],
'logics':[90,80,90,80]
}

print("\nDataFrame of student marks:")
pdmarks=pd.DataFrame(marks)
print(pdmarks)

print('\nLeaving only records with the 1-st class grade on analysis')
analysisfirst=pdmarks[pdmarks['analysis']>=70]
print(analysisfirst)
```

The output

```
DataFrame of student marks:
   id  algebra  analysis  logics
0  11       60        40      90
1  41       90        80      80
2  52       70        80      90
3   8       55        70      80

Leaving only records with the 1-st class grade on analysis
   id  algebra  analysis  logics
1  41       90        80      80
2  52       70        80      90
3   8       55        70      80
```

Durham University

A *Series* instance is not the same as a *DataFrame* instance with a single column

- *pdmarks['analysis']* produces a $*Series*$ instance.
- *L=['analysis']* and then *pdmarks[L]* produces a $*DataFrame*$ with a single *'analysis'* column.
- They are **not** the same objects.
- In particular, the latter needs application of the *squeeze()* functions, in case it used for filtering.

Durham University

Files .csv and Excel

From .csv file to a DataFrame and back to a .csv file

- *df=pd.read_csv('f.csv')* copies the content of *f.csv* file to a *DataFrame* instance *df*.
- *df.to_csv('f1.csv')* copies from *DataFrame df* to *f1.csv* file.
- Both operations preserve the columns of the dataset.

Example: reading from and writing to .csv file

The dataset has been downloaded from:
https://github.com/dsrscientist/dataset1/blob/master/student_marks.csv

The 1-st column has been named.

```
import pandas as pd

print("\nDataFrame of student marks created from .csv file:")
pdmarks = pd.read_csv('student_marks.csv')
print(pdmarks)

#Applying mean to the modules only restriction
pdmarks['Avmarks']=pdmarks[pdmarks.columns[3:]].mean(1)
pdavsorted=pdmarks.sort_values(by='Avmarks', ascending=False)
pdavsorted.to_csv('MarksAvSorted.csv',index=False)

print("\nNew .csv file is created")
```

Example: reading from and writing to .csv file

The input: *student_marks.csv*

	A	B	C	D	E	F	G	H	I	J	K
1	Name	Gender	DOB	Maths	Physics	Chemistry	English	Biology	Economic	History	Civics
2	John	M	05/04/1988	55	45	56	87	21	52	89	65
3	Suresh	M	04/05/1987	75	96	78	64	90	61	58	2
4	Ramesh	M	25/05/1989	25	54	89	76	95	87	56	74
5	Jessica	F	12/08/1990	78	96	86	63	54	89	75	45
6	Jennifer	F	02/09/1989	58	96	78	46	96	77	83	53
7	Annu	F	05/04/1988	45	87	52	89	55	89	87	52
8	pooja	F	04/05/1987	55	64	61	58	75	58	64	61
9	Ritesh	M	25/05/1989	54	76	87	56	25	56	76	87
10	Farha	F	12/08/1990	55	63	89	75	78	75	63	89
11	Mukesh	M	02/09/1989	96	46	77	83	58	83	46	77

The output: *MarksAvSorted.csv*

	A	B	C	D	E	F	G	H	I	J	K	L
1	Name	Gender	DOB	Maths	Physics	Chemistry	English	Biology	Economic	History	Civics	Avmarks
2	Jennifer	F	02/09/1989	58	96	78	46	96	77	83	53	73.375
3	Farha	F	12/08/1990	55	63	89	75	78	75	63	89	73.375
4	Jessica	F	12/08/1990	78	96	86	63	54	89	75	45	73.25
5	Mukesh	M	02/09/1989	96	46	77	83	58	83	46	77	70.75
6	Ramesh	M	25/05/1989	25	54	89	76	95	87	56	74	69.5
7	Annu	F	05/04/1988	45	87	52	89	55	89	87	52	69.5
8	Suresh	M	04/05/1987	75	96	78	64	90	61	58	2	65.5
9	Ritesh	M	25/05/1989	54	76	87	56	25	56	76	87	64.625
10	pooja	F	04/05/1987	55	64	61	58	75	58	64	61	62
11	John	M	05/04/1988	55	45	56	87	21	52	89	65	58.75

From Excel file to a DataFrame and back to Excel file

- Writing to an Excel file is straightforward:

  df.to_excel('f.xlsx') copies a *DataFrame* instance *df* to *'f.xlsx'* file.

- For reading, one possibility is to use *openpyxl* library.
  - Use *load_workbook* to create a workbook.
  - Make the workbook active (in our case "*marksactive*")
  - Create a *DataFrame* instance *df* by running *df=pd.DataFrame(marksactive.values)*,
    where "*marksactive"* is the active workbook.

Example mapping marks to their classes

- We will redo the same example we considered for working with *Excel* outside of Pandas.
- Instead of processing the cells one by one, mapping will be applied to columns using the function as below.

```
def marktoclass(mark):
  if 70<=mark:
    return  'First'
  elif mark in range(60,70):
    return 'Upper second'
  elif mark in range(50,60):
    return 'Lower second'
  elif mark in range(40,50):
    return 'Third'
  else:
    return 'Fail'
```

Example: mapping marks to their classes – main part

```
print("\nDataFrame of student marks created from Excel file:")
marksframe = openpyxl.load_workbook('StudentsMarks.xlsx')
marksactive=marksframe.active

pdmarks=pd.DataFrame(marksactive.values)
print(pdmarks)

#Creating a copy to leave the original sheet unchanged
pdclasses=pdmarks.copy()
for col in pdclasses.columns[1:]:
  pdclasses[col]=pdclasses[col].map(marktoclass)

pdclasses.to_excel('MarksClasses.xlsx')
print("\nNew Excel file is created")
```

Example: reading from and writing to .csv file

The input: *StudentMarks.xlsx*

	A	B	C	D	E
1	Abe	70	85	90	60
2	Carol	80	80	65	80
3	John	70	70	60	95
4	Mary	100	100	50	85

The output: *MarksClasses.csv*

	A	B	C	D	E	F
1		**0**	**1**	**2**	**3**	**4**
2	**0**	Abe	First	First	First	Upper second
3	**1**	Carol	First	First	Upper second	First
4	**2**	John	First	First	Upper second	First
5	**3**	Mary	First	First	Lower second	First

Durham University

Concluding remarks

Redundancies in Pandas

- **Important**: Pandas have a lot of redundancy. The same task can normally be solved in several different ways.
- There is no single correct answer of what approach is the best: all depends on the application and the choice of the developer.
- **Important**: always try the functionalities you want to use on simple examples

Approach change in the rest of the module

- So far, we have followed the 'Python oriented approach':
  considered various functionalities and examples,
  demonstrating how these functionalities can be used.

- In the remaining part of the module, we will follow the 'problem oriented approach':
  we will consider the data cleaning and analytics tasks and
  see how they can be solved within Python.